DAY 07 · BEGINNER · FREE API

Tools

Hand the model a Python function, then run the whole loop by hand so that nothing about it stays magic.

`@tool` `bind_tools` `tool_calls` `ToolMessage` `Notebook`

KGP Talkie
LEARN. BUILD. TALK AI

Scan to subscribe

Laxmi Kant Tiwari · kgptalkie.com

The 10-Day Course Map

WEEK 1 TALKING TO A MODEL

- 01 **Your First AI Program** — Ask a model, print the answer
- 02 **Roles and Messages** — The system message sets the job
- 03 **Prompt Templates** — One template, many inputs

YOU WILL HAVE BUILT
a script that answers anything you type into it

WEEK 2 BUILDING SOMETHING

- 04 **Chains: The Pipe** — `prompt | model | parser`
- 05 **Structured Output** — An object, not a paragraph
- 06 **Memory** — A checkpointer and a thread id

YOU WILL HAVE BUILT
a chatbot that remembers what you said a minute ago

WEEK 3 GIVING IT ABILITIES

- 07 **Tools** — Your Python, called by the model — YOU ARE HERE
- 08 **Your First Agent** — It decides when to call them
- 09 **Chunks and Vectors** — Find a paragraph by meaning
- 10 **RAG with RAGWire** — Citations, and a refusal

One notebook and one deck a day, **60 to 90 minutes** each. Every model used is a **free** one, and Docker only turns up on Day 9.

Why the Model Needs Tools

LIVE DATA

"What is the USD to INR rate right now?"

Trained, then frozen.

Today's rate was never in it.

an honest refusal

YOUR DATA

"Zephyr Airlines baggage allowance?"

A made-up airline, in a file

on your own disk.

Day 10 fixes this one

Tools are not about making the model cleverer. They connect it to **things that are true right now**.

What the Model Sees in a Tool

YOUR FUNCTION

```python
@tool
def to_rupees(
amount: float, currency: str
) -> float:
"""Convert an amount from USD,
EUR or GBP into Indian rupees."""
return amount * RATES[currency]
```

the body is never sent



WHAT THE MODEL RECEIVES

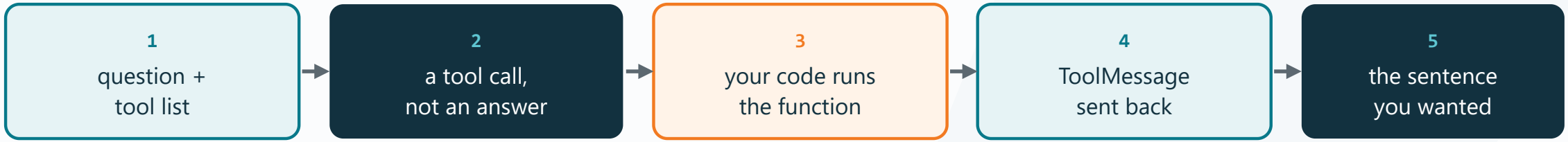
name
"to_rupees"

description (your docstring)
"Convert an amount from USD,
EUR or GBP into Indian rupees."

args (your type hints)
amount: number, currency: string

The docstring is not documentation. It is the **interface**.

The Tool-Calling Loop, Step by Step



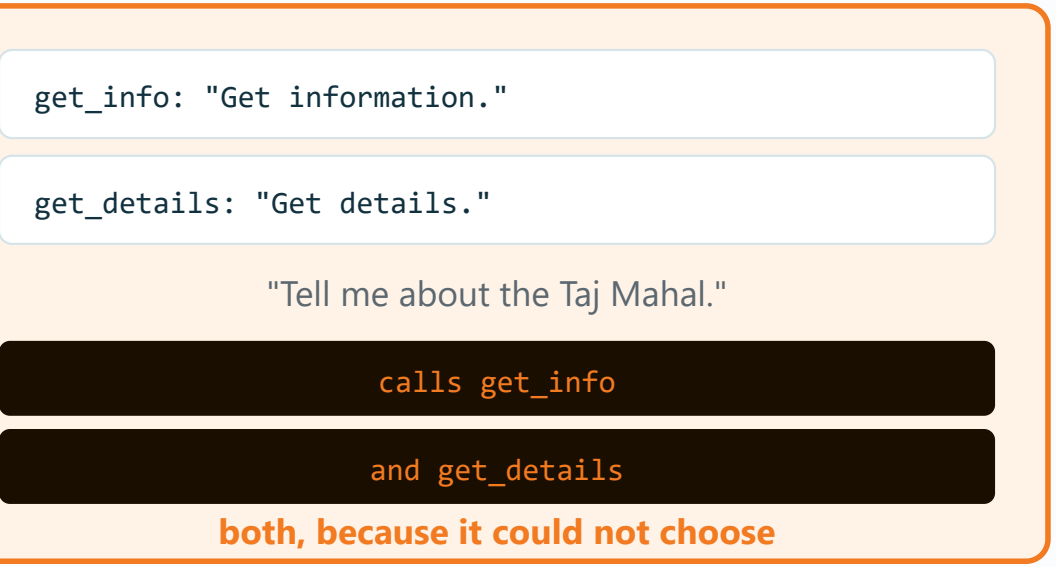

```
ToolMessage(content='22100.0', name='to_rupees', tool_call_id='call_7e6e...')
```

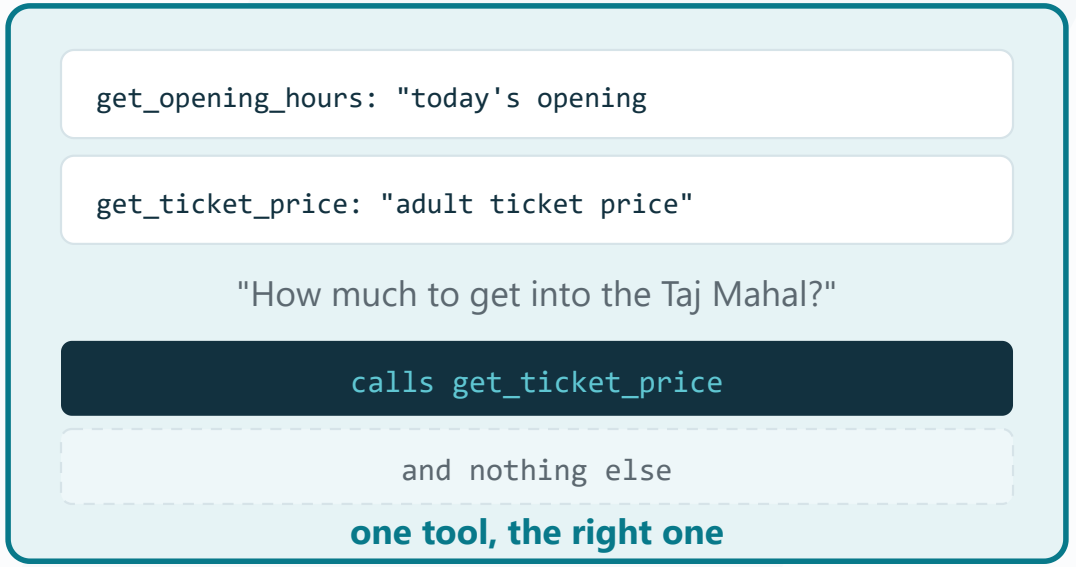
the id pairs the reply with the request

The model asks. **Your code runs it.**
Every agent framework is a wrapper around those five steps.

Tomorrow you hand exactly this loop to `create_agent`.

Writing Good Tool Descriptions





Nothing about the code changed. **The fix was a sentence.**

CREATE_AGENT RUNS THIS LOOP FOR YOU

Tomorrow: Your First Agent

TODAY, BY HAND

```
reply = with_tools.invoke(question)
call = reply.tool_calls[0]
result = to_rupees.invoke(call)

final = with_tools.invoke([
    HumanMessage(question),
    reply,
    result,
])
```

> You decided there was **one** round trip. What if it needs two?

TOMORROW

```
agent = create_agent(
    model=model,
    tools=[to_rupees, multiply],
)

agent.invoke({"messages": [...]})
```

- The **model** decides how many times round the loop.
- Every step is still a message you can read.
- And one unhandled exception takes the whole thing down.